Airport Investment Radar - Design Document

1. Purpose

Airport Investment Radar is an AI-assisted research tool for airport investment analysts.

The product helps analysts identify US airports that may be strong candidates for modernization, terminal expansion, or capacity investment. It combines public aviation data, deterministic scoring, live aviation signals, and AI-generated explanations in a conversational interface.

The core business question is:

> ***Which airports show signs that infrastructure investment could unlock unmet flight or passenger capacity?***

This is not a final ROI model. It is a first-pass screening system that helps analysts quickly discover which airports deserve deeper investigation.

2. Current Analyst Pain Points

Today, an analyst who wants to evaluate airport modernization opportunities usually needs to do the work manually:

1. Search across multiple aviation data sources.
2. Find airport metadata, runway information, demand forecasts, route data, and flight activity.
3. Normalize data from different formats such as CSV, XLSX, and APIs.
4. Calculate comparable metrics across airports.
5. Build a ranking or comparison manually.
6. Explain the reasoning in a way that is useful for investment discussion.
7. Repeat the same process for every follow-up question.

This can take hours for a single region or airport comparison. The bigger issue is not only time, it is consistency. Different analysts may use different assumptions, different sources, or different formulas.

Airport Investment Radar solves this by giving analysts one place to ask airport investment questions, receive structured scoring, understand the reasoning, and continue with follow-up questions without restarting the research process.

3. What the Agent Solves

Airport Investment Radar turns fragmented aviation research into a faster, repeatable workflow.

It helps analysts:

- rank airports in a region by investment potential
- compare congestion and capacity pressure between airports
- estimate long-haul opportunity
- estimate unmet demand using transparent proxy metrics
- understand why one airport scored higher than another
- ask follow-up questions using saved conversation history
- see assumptions and uncertainty instead of hidden black-box reasoning

The value is not that the agent replaces the analyst. The value is that it gives the analyst a reliable first-pass research assistant that collects signals, calculates metrics, and explains the result clearly.

4. Core Principle

> ***AI explains. Backend calculates. Data sources provide evidence.***

Claude is used as the reasoning and explanation layer. It understands the analyst question, chooses the right tool, and explains the structured result.

The backend is the source-of-truth layer. It fetches and normalizes data, calculates deterministic scores, caches results, and returns structured evidence to Claude.

Claude never invents airport metrics and never calculates final scores. Every score, number, and ranking comes from backend tools.

5. Assumptions

Business Assumptions

- The analyst is looking for early-stage investment opportunities, not a final investment decision.
- The most valuable first-pass signal is whether an airport appears capacity-constrained while demand is stable or growing.
- Higher passenger demand, higher flight activity, stronger long-haul share, and visible congestion pressure may indicate stronger modernization potential.
- Public data does not fully expose terminal-level constraints such as gate count, baggage capacity, security capacity, or renovation cost.
- The score is used to prioritize deeper research, not to approve investment.

Technical Assumptions

- OurAirports and FAA TAF files are treated as local public datasets because they are mostly static reference data and do not need live API calls on every request.
- AeroDataBox via RapidAPI is used only where fresh route or flight activity signals are useful.
- API responses are cached because live aviation APIs are paid or limited by free-tier quotas. Caching reduces cost, protects request limits, and improves response time.
- PostgreSQL stores dynamic application data, not full public datasets.
- Conversation history is stored on the server so follow-up questions can be answered reliably.
- Airport scores are cached as snapshots because repeated follow-up questions often reuse the same analysis.
- Tool-call logs are stored so each answer can be audited and traced back to the data and tools used.

6. Architecture

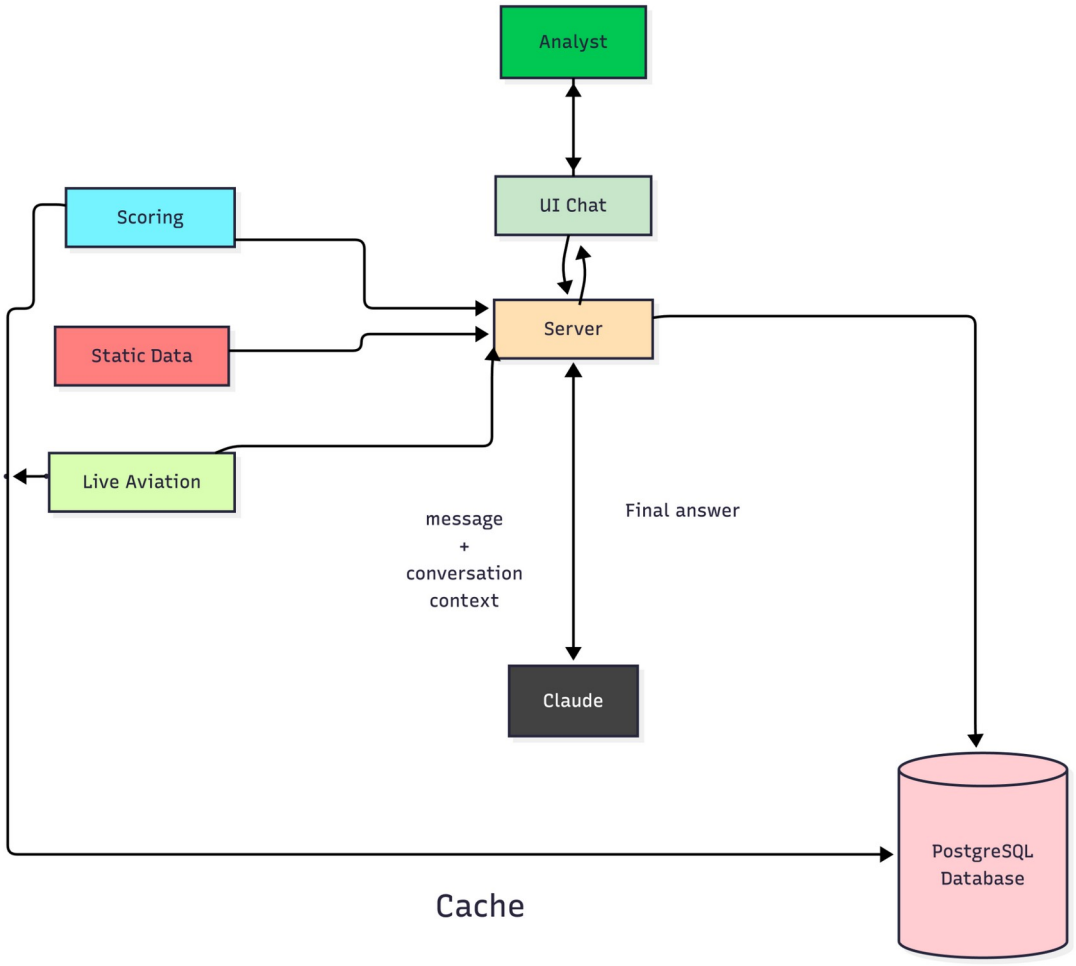

Figure 1: Airport Investment Radar agent flow

Simple flow:

1. The analyst asks a question in the Angular chat UI.
2. The NestJS server sends the message and conversation context to Claude.
3. Claude decides whether it needs data and requests a backend tool.
4. The server executes the tool through the tools layer.
5. The tool can use scoring logic, static data files, AeroDataBox live aviation data, and PostgreSQL.
6. The server returns structured tool results back to Claude.
7. Claude produces the final analyst-facing answer.
8. PostgreSQL stores conversation history, API cache, score snapshots, and tool-call logs in the background.

The agent loop is limited to 8 iterations to prevent infinite tool-calling loops.

7. Main Analyst Workflows

Rank airports in a region

Example: Which airports in New England are strong candidates for terminal expansion?

The system resolves the region, scores candidate airports, ranks them, and explains the top candidates.

Compare two airports

Example: Compare LA and Santa Ana airport congestion levels.

The system resolves the airports, compares congestion and demand signals, and explains which airport appears more capacity-constrained.

Analyze long-haul share

Example: What is the percentage of long-haul flights out of Anchorage airport?

The system uses route data and distance calculation to estimate long-haul share.

Estimate unmet demand

Example: What is the unmet flight demand in SFO airport and why?

The system combines demand growth, congestion pressure, and capacity constraints into a proxy-based unmet demand explanation.

Follow-up questions

Example: Why did BOS rank higher than Providence?

The backend stores conversation history, so the analyst can continue the discussion without repeating context.

8. Scoring Methodology

Every airport receives a deterministic Investment Score from 0 to 100.

The score is intended for first-pass screening, not final ROI prediction. Claude does not calculate or modify scores. The backend scoring engine calculates every number, and Claude only explains the result.

```
Investment Score =
  35% Congestion Pressure
+ 25% Activity Demand
+ 20% Long-Haul Opportunity
+ 20% Unmet Demand Proxy
```

Dimension	What it answers	Weight
Congestion Pressure	Is this airport already under infrastructure pressure?	35%
Activity Demand	Is the airport large enough, and is demand growing?	25%
Long-Haul Opportunity	Does the airport justify higher-value international or long-haul infrastructure?	20%
Unmet Demand Proxy	Is growing demand hitting existing capacity constraints?	20%

Congestion Pressure, 35%

Measures commercial demand relative to runway infrastructure.

```
50% operations per runway
30% annual air carrier itinerant operations
20% delay / cancellation signal
```

This is better than runway count alone. A major airport with four runways and hundreds of thousands of yearly operations can be more constrained than a small airport with two lightly used runways. Operations per runway captures utilization, while annual air carrier itinerant operations ensures the airport is large enough to matter for investment.

Activity Demand, 25%

Measures current commercial scale and future demand.

```
50% current air carrier enplanements from FAA TAF
25% enplanement growth forecast
```

```
25% operations growth forecast
```

Current air carrier enplanements show the size of today's passenger market. FAA forecast growth shows whether demand is expected to increase. Together, they help identify airports where modernization has a meaningful demand base.

Long-Haul Opportunity, 20%

Measures the share of departing routes that are long-haul.

A route is considered long-haul when the distance is at least 3,000 km. The score is normalized so that airports with 30% or more long-haul routes receive full marks for this dimension.

Long-haul routes matter because they usually require higher-value infrastructure: international terminals, customs and immigration capacity, wide-body gates, premium lounges, baggage handling, and stronger passenger processing.

Unmet Demand Proxy, 20%

Estimates whether demand is growing faster than the airport can comfortably handle.

```
50% growth trajectory
30% operations-per-runway pressure
20% congestion score
```

This dimension captures the main investment signal: growth meeting constraint. An airport that is congested but not growing may be an operational issue. An airport that is growing but uncongested may still have room to absorb demand. An airport that is both growing and constrained is a stronger modernization candidate.

Score Meaning

Score	Grade	Meaning
80+	A	Strong investment signal. Immediate due diligence candidate.
65-79	B	Clear investment case. Add to shortlist.
50-64	C	Moderate case. May fit specific infrastructure plays.
35-49	D	Weak signal. Monitor but do not prioritize.
Below 35	F	No meaningful investment signal in this screening model.

Important Limitation

The score is a screening and prioritization tool. It does not measure land availability, regulatory constraints, ownership structure, project cost, airport revenue, or full financial feasibility. Those require deeper investment analysis.

9. Why the Formula Matters

The formula is what makes the agent trustworthy.

Without deterministic scoring, the answer would be only an LLM opinion. With deterministic scoring:

- airports are compared consistently
- rankings are reproducible
- analysts can see which factor drove the result
- assumptions can be challenged or adjusted
- scores can be cached and audited
- future data sources can improve the model without changing the product flow

The AI explains the score. The backend owns the score.

10. Data Strategy

Airport Investment Radar uses a hybrid data strategy.

The system combines static public datasets, cache-first live aviation APIs, and PostgreSQL persistence. The goal is to provide useful investment signals while keeping the product reliable, explainable, and cost-aware.

Local public datasets

Source	Usage	Reason
OurAirports CSV	airport metadata, coordinates, runway count, runway length	stable public data, fast local lookup, no API quota
FAA TAF XLSX	operations, enplanements, forecast growth	official aviation planning data for demand signals

These files are loaded once at startup into memory. They are not stored fully in PostgreSQL because they are static reference datasets.

Live API, cache-first

Source	Usage	Cache TTL
AeroDataBox via RapidAPI departures endpoint	live departures, delay rate, cancellation rate	24 hours
AeroDataBox via RapidAPI daily routes endpoint	route network and long-haul share calculation	7 days

AeroDataBox is used as the live aviation data provider. The backend uses a departures endpoint for live flights, delays, and cancellations, and a daily routes endpoint for route statistics and long-haul route analysis.

All AeroDataBox responses are cached in PostgreSQL before being reused. This reduces latency, protects free-tier quota, lowers API cost risk, and makes repeated follow-up questions reliable.

Missing Data Handling

Small and regional airports often have less complete data, especially for live routes and delay signals. The system handles missing data explicitly and surfaces uncertainty in the final answer.

Scenario	Handling	Output note
Delay signal unavailable	Default to 0, which may understate congestion	Delay data unavailable, congestion may be understated
Route destination coordinates missing	Treat unresolved destination as long-haul when no coordinates are available	Some destinations were assumed long-haul
FAA TAF forecast missing	Growth components default to the floor	FAA TAF data missing, growth score is limited
Airport not found in reference dataset	Reject the query with a clear explanation	Airport not found in reference dataset

If AeroDataBox is unavailable or not configured, the delay sub-score and long-haul score may default to 0. This can understate scores for high-volume or internationally connected airports. The system surfaces this uncertainty in the response instead of hiding it.

Scores for well-documented airports are more reliable than scores for small airports with incomplete public data. The system communicates this difference instead of hiding it behind a single number.

The system does not silently substitute a median value. Every fallback is disclosed with a plain-language explanation of what was missing and what assumption was used.

11. Persistence and Caching

PostgreSQL is used for dynamic data only.

Table	Purpose
Conversation and Message	stores chat history for follow-up questions
ApiCache	stores live API responses with TTL
AirportScoreSnapshot	stores calculated airport scores for reuse
ToolCall	stores tool name, input, output, duration, and conversationId

Why this matters:

- follow-up questions work reliably
- limited API requests are not wasted
- repeated airport analysis is faster
- every answer can be traced back to tool calls and data sources

12. Where AI Is Used

AI is used for:

- understanding analyst questions
- selecting the right backend tool
- turning structured scoring results into readable explanations
- supporting follow-up questions
- communicating assumptions and uncertainty clearly

AI is not used for:

- calculating scores
- inventing airport statistics
- accessing APIs directly
- deciding rankings without backend evidence

13. Key Tradeoffs

Tradeoff	Decision	Reason
Perfect investment model vs useful screening tool	Build a transparent screening score	achievable, explainable, and useful for early analysis
Static data vs live APIs	Use local FAA and OurAirports data plus cache-first live API	stable demo with useful live signals
Live API freshness vs free-tier limits	Cache API responses with TTL	protects paid or limited API usage while keeping useful signals
One-day scope vs production architecture	Build the core agent loop first	proves the product value without overbuilding
LLM scoring vs backend scoring	Backend calculates all scores	reproducible, auditable, and trustworthy
PostgreSQL vs separate cache system	Use PostgreSQL for cache and history	simpler architecture and enough for this prototype
Client-sent history vs server history	Server stores authoritative history	better follow-up support and avoids history drift

14. Future Improvements

Because this version was built as a one-day prototype, it focuses on the core value: a working agent, deterministic scoring, explainable results, and a usable chat interface. The next production steps would be:

- Move cache to Redis: PostgreSQL is sufficient for the prototype, but Redis is better for high-frequency TTL cache reads, faster API response reuse, and score lookup at scale.
- Add WebSocket or streaming responses: The current REST flow is simple and reliable. A production version could stream tool progress and partial responses so the UI feels faster and can always listen for updates.
- Add authentication and authorization: Login/logout and per-user authorization would allow each analyst to see only their own chat history, saved analyses, and organization-specific data.
- Use a unified paid aviation data provider: A paid provider that aggregates route, flight, capacity, passenger, and delay data would reduce reliance on static files and free-tier APIs, improve coverage, and simplify data normalization.
- Route between Claude models by task: Use a faster or cheaper model for simple tool routing, and a stronger model for complex comparisons, final investment explanations, or multi-step reasoning.

- Add richer investment signals: Include land availability, terminal gates, airport master plans, slot controls, noise restrictions, concession structure, CAPEX estimates, revenue, and cargo/passenger weighting.
- Add scheduled data refresh jobs: Precompute airport scores nightly, refresh API cache in the background, and alert analysts when airport scores materially change.
- Add E2E tests: Build end-to-end tests that run the full user journey across client, server, database, tools, scoring, and cache behavior. This would help keep the service healthy and prevent changes from breaking the agent flow.
- Add evaluation and monitoring: Create a benchmark set of questions, track tool success rate, latency, API usage, score stability, and answer quality over time.
- Add analyst export and visualization: Provide map views, score breakdown charts, comparison tables, and exportable investment briefs for sharing with stakeholders.

15. Success Criteria

The project is successful if an analyst can:

1. ask airport investment questions in a chat interface
2. rank airports by region
3. compare two airports
4. analyze long-haul share
5. estimate unmet demand with clear caveats
6. ask follow-up questions using conversation history
7. understand the scoring logic and data sources behind each answer

16. One-Sentence Summary

Airport Investment Radar helps analysts identify airport modernization opportunities by replacing hours of manual data gathering and metric calculation with a conversational agent that combines local FAA and OurAirports data, AeroDataBox live aviation data via RapidAPI, cache-first API usage, deterministic backend scoring, and Claude-generated investment explanations.